02-学习记录

一、计算机的存储规则

文本数据、image图片、sound声音

文本数据：数字、字母、汉字

计算机使用的存储方式：任意数据都是二进制形式存储的：0和1组成。

二、计算机常用进制

2.1、十进制、二进制、八进制、十六进制

2.1.1、十进制：0123456789

逢十进一、借一当十

2.1.2、二进制：由0和1

逢二进一、借一当二

十进制转二进制：

数字：2->10; 3->11; 4->100

2.1.3、八进制：01234567

逢八进一

2.1.4、十六进制：0123456789abcdef

逢十六进一

2.2、不同进制在代码中的表现形式：

二进制：有0和1组成，代码中以0b开头

十进制：有0~9组成，不加任何前缀

八进制：由0~7 组成，代码中 0 开头

十六进制：由0~9 还有a~f 组成，代码中以0x开头

2.3、任意进制转十进制

公式：系数（每一位上的数） * 基数（进制数）的权次幂 相加

系数： 就是每一位上的数字

基数：当前进制数、

权：从右往左，依次为0 1 2 3 4 5 6 ...

例如：二级制：转为十进制

1 0 1

计算过程：系统* 基数的权次幂

1*2^2 + 0*2^1 + 1*2^0

4 + 0 + 1 结果：对应十进制等于：5

八进制:转为十进制

1 0 1

计算过程：系统* 基数的权次幂

1*8^2 + 0*8^1 + 1*8^0

64 + 0 + 1 结果：对应十进制等于：65

十六进制:转为十进制

a b c

计算过程：系统* 基数的权次幂

10*16^2 + 11*16^1 + 12*16^0

2560 + 176 + 12 结果：对应十进制等于：2748

2.4、二进制转十进制：

计算公式：8421快速转换法

每一位二级制值的1都是代表一个固定数值

把每一位的1代表的十进制数加起来得到的结果就是它所代表的十进制数。

例如：

1	1	1	1	1	1	1	1	
128	64	32	16	8	4	2	1	
			例如：二进制 1101					结果
				1	1	0	1	13

2.5、十进制转其他进制

除基数取余数

不断的除以基数（几进制，基数就是几）得到余数，直到商为0，再将余数倒着拼接起来即可。

例如：十进制11转为2进制			
2	11		实际结果：1011
2	5（商）	1（余数）	倒着拼接
2	2（商）	1（余数）	
2	1（商）	0（余数）	
	0（商）	1（余数）	

2.6、ascii码表

存储：数字和字母

字母查询ascii码表 例如：a->97 （十进制数）计算机最终存储的是 **97 对应的二进制形式 (01100001)**

1、gb2312: 1981年5月1日 简体中文汉字，收录7445个图形字符，包括6763个汉字

2、big5 ： 13053个中文字、1984年

3、gbk ： 2000年3月17日发布，收录21003汉字，包括GB13000-1的全部中日韩汉字，包括BIG5中的所有汉字。

4、Unicode编码：国际标准字符集、它将世界各种语言的每个字符定义一个唯一的**数字**，以满足跨语言、跨平台的文本信息转换。

5、汉字在计算机中的存储流程是：汉字 -> 查询码表（如 GBK/Unicode）得到对应的数字 -> 将该数字转换为二进制存储。

三、图片数据：

通过每一个像素点中的RGB三原色来存储的。

图片数据-灰度图 0~255表示灰度范围

图片数据-黑白图 -》 0 黑； 1 白

通常在计算机中，**0 代表黑色（无光/无墨）**，**1 代表白色（有光/有墨）**。特别是在早期的位图定义中，0 通常表示关闭（黑），1 表示开启（白）。不过这一点在不同系统中可能有差异，但通常 0 为黑是主流（如纯黑 RGB(0,0,0)）

图片数据--彩色图

每个像素的颜色由 **RGB 三个分量**组成，每个分量的取值范围是 0~255，因此总共可以表示 256×256×256256×256×256 种颜色

三基色-三原色

计算机中的颜色采用光学三原色 RGB

红绿蓝 取值范围分别都是： 十进制0~255 或者 0~FF FF FF

四、声音数据

波形图，对声音的波形图进行采样再存储的。

五、java的数据类型：基本数据类型、引用数据类型

基本数据类型的四类八种

整数：byte（-128~127）、short（-32768~32767）、int 占用 **32 位（4 字节）**，其数值范围约为 **-21 亿 到 +21 亿**。虽然最大值 2147483647 确实是 10 位数，但通常我们强调其位数或字节大小、long（-92233720368547**7904**~9223372036854775807（19位数））

浮点数：float（-3.401298e-38 到3.402823e+38）、double （-4.9000000e-324到1.797693e+308）

字符：char（0~65535）是基于 Unicode 字符集的，这与前面的字符编码部分是呼应的。

布尔：boolean （true/false）

byte的取值范围

整数和小数取值范围大小关系：

double>float>long>int>short>byte

六、标识符：就是给类、方法、变量等起的名字

七、键盘输入：Scanner

八、运算符：

8.1、运算符：对字面量或者变量进行操作的符号

8.2、表达式

用运算符把字面量 或者变量连接起来，符合java语法的式子就可以成为表达式。不同运算符连接的表达式体现的是不同类型的表达式。

例如：

int a=10;

int b=20;

int c= a + b;

上面例子：+ 是运算符 并且是算术运算符、a+b 是表达式 ，由于 + 是算术运算符，所以这个表达式叫算术表达式。

---需要学习目录---：

1、算术运算符、2、自增自减运算符、3、赋值运算符、4、关系运算符、5、逻辑运算符、6、三元运算符。运算符优先级等。

1.1、算术运算符：

加：+ 、减：-、乘： * （shift +8）、除： /

取余/取模： % （难点）

取余 和 取模 应用场景：

1.可以用取模来判断，A是否可以被B整除

A%B 10%3

2.可以用来判断A是否为偶数

A % 2,如果结果为0，那么证明A是一个偶数，否则为一个奇数。

3.斗地主：三个玩家，把每一个张牌都定义一个序号，拿着序号 % 3 如果结果为1，就给第一个玩家，如果 结果为2 ，那么就发给第二个玩家，如果结果为0，那么就给第三个玩家。

/ 和 % 区别 ： 两个数据做除法，/ 取结果的商，% 取结果的余数。

整数操作，只能得到整数，要想得到小数，必须有浮点数参与计算。

------1.1.1、学习 “+” 三种情况。

包括：数字相加、字符串相加、字符相加。

数字相加：

会涉及：隐式转换 和 强制转换

隐式转换：取值范围小的数值，转为大的取值范围。

隐式转换（自动类型提升）

把一个取值范围小的数值，转为取值范围大的数据。

取值范围：

double>float>long>int>short>byte

什么时候转换?

数据类型不一致，不能进行计算，需要转成一样的才可以计算。

转换规则1：

取值范围小的和取值范围大的进行运算，小的会提升为大的，再进行计算

转换规则2：

byte/short/char 三种类型的数据 在运算的时候，都会直接先提示int 在计算。

强制转换：取值范围大的数值，转为小的取值范围。

1.如果把一个取值范围大的数据，赋值给取值范围小的变量，是不允许的直接赋值的，如果一定要执行的话，就需要加入强制转换了。

2.格式：目标数据类型 变量名 =（目标数据类型） 被强制的数据。

字符串相加：

当 “+” 操作中出现字符串时，这个 + 是字符串连接符，而不是算术运算符了，会将前后的数据进行拼接，并产生一个新的字符。

例如： result = “123” +123 问？ 246 / 123123

连续进行 + 操作时，从左到右逐个执行。

例如： 1+ 99 + “听说能变强” 问？ 100听说能变强 /199听说能变强。

字符相加：

当 字符 + 字符 /字符+数字 时，会把字符通过ASCII码表查询到数字，再计算。

例如： 1 + ‘a’ // 98/ 1a 98。

'a'+"abc" // abc /97abc ?

注意：

(1) byte,short分别与char两两之间转换，不能使用自动类型转换。

(2) byte,short分别与char两两之间转换，只能使用强制转换。但是这样很容易造成精度丢失，导致转换数据不准确!!!因为他们之间转换，都会首先转换为int类型，再转换为他们之间的其他类型数据。然而他们本身存储数据范围很小，大于128就会转换出错。所以谨慎使用!!!

(3) byte与short之间，既可以使用自动类型转换，也可以使用强制类型转换。因为他们本质都属于整型数据，只是存储数据范围不同。

(4) boolean类型不能与其它数据类型运算，否则报错。

(5) String类型是引用类型数据，也不能与这8种基本数据类型转换，否则报错。

2、自增自减运算符：

基本用法：

自增自减运算符		
基本用法		
符号	**作用**	**说明**
++	加	变量的值加1
--	减	变量的值减1

注意事项：

++ 和 - - 既可以放在变量的前面，也可以放在变量的后边。

int a =10;

++a; 或者 a++;

System.out.println(a);//11

单独使用: ++ 和 - - 无论是放在变量的前面还是后边，单独写一行的运行结果是一样的。

参与计算:

场景1： int a=10; int b=a++; (先用后加)

场景2： int a=10; int b=++a; (先加后用)

3、赋值运算符

赋值运算符		
符号	作用	说明
“=”	赋值	int a=10 ，将10 赋值给 a
+=	加后赋值	a+=b，将a+b的值，赋值给a
-=	减后赋值	a-=b，将 a-b的值，赋值给a
*=	乘后赋值	a*=b，将 a×b的值，赋值给a
/=	除后赋值	a/=b，将 a÷b的商，赋值给a
%=	取余后赋值	a%=b，将 a÷b的余数，赋值给a

+= ：规则：将左边和右边进行相加，然后再把结果赋值给左边。

注意事项：扩展的赋值运算符隐含了强制类型转换。

4、关系运算符/比较运算符的分类

关系运算符/比较运算符	
符号	说明
“==”	a==b，判断a和b是否相等，成立为true，不成立为false
!=	a!=b，判断a和b是否不相等，成立为true，不成立为false
>	a>b，判断a是否大于b，成立为true，不成立为false
>=	a>=b，判断a是否大于等于b，成立为true，不成立为false
<	a<b，判断a是否小于b，成立为true，不成立为false
<=	a<=b，判断a是否小于等于b，成立为true，不成立为false

注意事项：关系运算符的结果，都是boolean类型，要么为true，要么为false，并且不要把 “==” ，误写成“=” 。

5、逻辑运算符

※ 在数学学习中，一个数据x，大于5，小于15，我们可以这样表示：5 < x < 15;

※ 在java中，是需要把上面的式子，先进行拆分，再进行合并表达式。

拆分为 ： x > 5 和 x < 15

合并后：x > 5 & x < 15 //并且

逻辑运算符		
符号	作用	说明
&	逻辑与 （且）	并且，两边都为真，结果才为真
|	逻辑或	或者，两边都为假，结果才是假
^	逻辑异或	相同为false，不同为true
!	逻辑非	取反

5.1、短路运算符 ： 。&& 和 ||

短路逻辑运算符		
符号	作用	说明
&&&	短路与	结果和&相同，但是有短路效果
||	短路或	结果和|相同，但是有短路效果

注意事项：

※ & 、| ，无论左边 true ， false，右边都要执行。

&& 、 || ：如果左边能确定整个表达式的结果，右边不执行。

&& ：左边为false，右边不管是真是假，整个表达式的结果一定是false。

||：左边为true，右边不管是真还是假，整个表达式的结果一定是true。

这两种情况下，右边不执行，提高了效率。

※ 实际开发中-工作中 ，用的最多还是逻辑运算符： && ， || ， !

6、三元运算符（三目运算符）：

格式：关系表达式 ? 表达式1 : 表达式2 ;

例如： 求两个数的较大值。

int max = a > b ? a : b ;

计算规则：

首先计算关系表达式的值，如果值为true，表达式1的值就是运算结果，如果为false，表达式2的值就是运算结果。

6.1、运算符的优先级：

优先级	运算符	结合性	描述
1 (最高)	`()`，`[]`，`.`，`::`	左到右	括号、数组下标、成员访问、方法引用
2	`++`，`--`，`+` (一元)，`-` (一元)，`~`，`!`，`(type)`	右到左	自增、自减、一元正负、按位取反、逻辑非、强制类型转换
3	`*`，`/`，`%`	左到右	乘、除、取模
4	`+`，`-`	左到右	加、减（以及字符串连接）
5	`<<`，`>>`，`>>>`	左到右	左移、有符号右移、无符号右移
6	`<`，`>`，`<=`，`>=`，`instanceof`	左到右	关系比较、类型检查
7	`==`，`!=`	左到右	相等性比较
8	`&`	左到右	按位与
9	`^`	左到右	按位异或
10	`|`	左到右	按位或

11	`&&`	左到右	逻辑与 (短路与)
12	`||`	左到右	逻辑或 (短路或)
13	`? :`	右到左	三元条件运算符
14	`=`, `+=`, `-=`, `*=`, `/=`, `%=` ...	右到左	赋值运算符
15 (最低)	`,`	左到右	逗号分隔符

下个switch